2025학년도 2학기 C프로그래밍 기말 프로젝트 보고서 안내

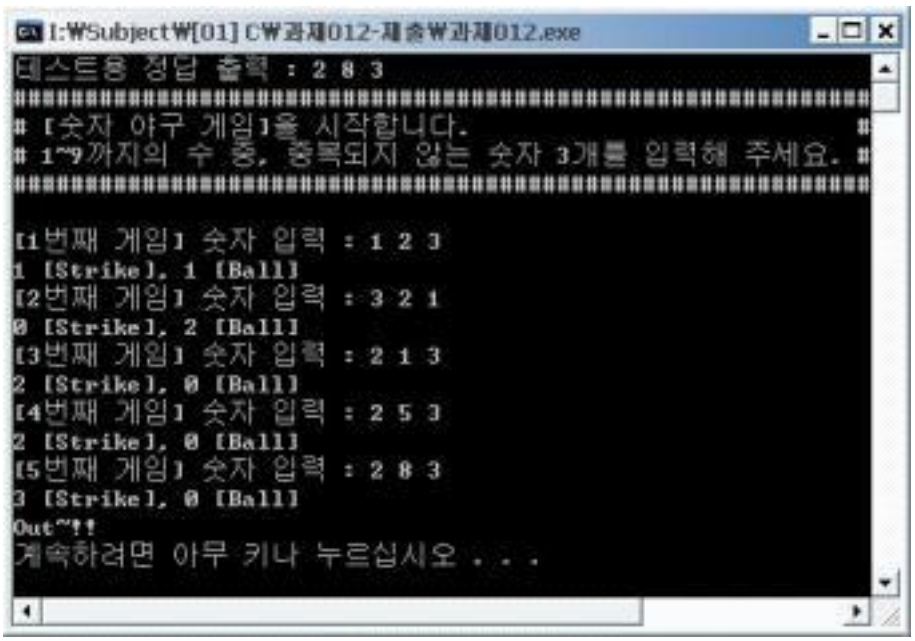
담당교수 : 김병창

- 기말 프로젝트의 난이도는 각각 상중하로 구분되어 있으며, 각 난이도 별로 차등 채점을 할 것임.
- 낮은 난이도라 할지라도 상위 난위도에 비하여 프로그램의 작성 난이도나 완성도, 보고서 작성등에서 월등하다면 난이도에 상관없이 최고 점수를 받을 수도 있음.
- **수업 교재의 부록에는 총 11개의 프로젝트 주제가 수록되어 있다. 또한 아래의 9개 프로젝트 주제가 소개되어 있다. 이 중 자신이 도전하고자 하는 주제를 선정하여 프로젝트를 진행하시오. 소개되지 않은 다른 자유 주제를 선택하여 진행하는 것도 가능함. (단, 중급 이상의 난이도가 있는 주제여야 함.)**
- 보고서는 반드시 논문 형식을 따라 작성하여야 하며 다음의 순서를 엄수해야 함.
  - 제목, 학번 및 이름, 소속 학과 및 학교, 요약
  - 1. 서론, 2. 관련 연구, 3. 프로그램의 설계, 4. 프로그램의 구현, 5. 프로그램의 동작과 결과, 6. 결과 분석 및 토의, 7. 결론, 8. 참고문헌
  - 별첨. (소스코드 일체와 주석 및 추가 첨부 자료)
- 보고서의 제출 기한 이후에는 일절 제출을 받지 아니함. 제출은 강의지원시스템에 전체를 압축한 파일과 보고서를 함께 제출토록 함.
- 모든 프로그램의 내용은 본인이 작성한 것이어야 하며, 본인이 모든 저작권을 소유하여야 함. 단, 부득이한 경우 타인 혹은 웹상의 공개 프로그램을 이용할 수 있으나, 반드시 타인 혹은 웹 상의 저작권을 명시하고, 참고문헌에 기록해야 하며, 본인이 수정 보완한 부분에 대한 명확한 언급이 있어야 함.
- 인공지능(AI, LLM) 서비스 사용가능하며, 그런 경우에는 어떤 prompt를 입력하였는지 참고문헌에 명시하여야 함.

기말 프로젝트 문제

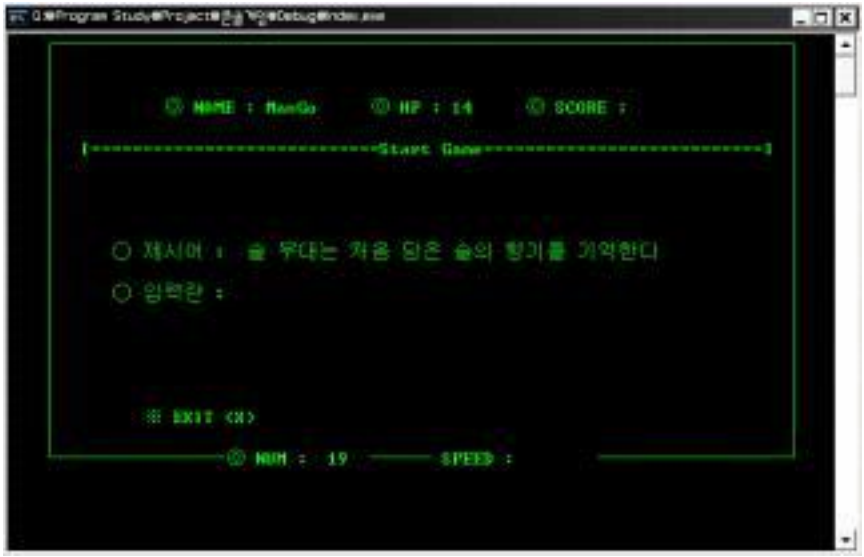
1. [난이도 下] 행맨(HangMan) 게임을 만드시오. 한 번의 게임 진행 후에 게임을 계속할 것인지를 물어보지 말고 실패할 때까지 다음 게임이 진행되도록 하고, 실패하면 그동안 성공한 게임의 수와 점수를 계산하여 출력한 후 게임을 계속할지 물어보도록 작성하시오. 이때 게임의 점수는 각 개인이 정책을 결정하여 채점하도록 하시오. 개인의 다양한 아이디어를 게임에 추가하는 것을 권장함.
   참고 : https://namu.wiki/w/행맨
   https://www.google.co.kr/search?q=행맨+게임&client=opera&hs=NAO&source=lnms&tbm=isch&sa=X&ved=0ahUKEwi3sZ7i54LUAhWHkZQKHfwyANwQ_AUIBigB&biw=1268&bih=1348

2. [난이도 下] 숫자로 하는 야구 게임을 만드시오. 컴퓨터가 임의의 세 숫자를 생성하고(사용자에게 모이지 않음), 사용자는 그 세 자리 숫자를 맞추는 게임이다. 위치와 숫자를 맞추면 스트라이크, 숫자만 맞추면 볼이며, 세 자리의 숫자는 중복되지 않는 1~9 사이의 숫자이다. 삼진이 되면 사용자가 이기는 것이고, 세 번 이상 노스트라이크 노볼이면 컴퓨터가 이긴다. 게임은 한번 실행으로 사용자가 원하는 만큼 할 수 있으며, 사용자가 종료 명령을 내리기 전까지 종료 되어서는 안 된다. 그 외의 다양한 기능을 작성자가 임의로 추가 할 수 있으며, 그에 따른 내용을 보고서에 명시하여야 한다.
   참고 : https://namu.wiki/w/숫자야구
   https://www.google.co.kr/search?q=숫자+야구&client=opera&hs=EBO&source=lnms&tbm=isch&sa=X&ved=0ahUKEwiMy-7754LUAhXIspQKHVq3Bw0Q_AUIBygC&biw=1268&bih=1348



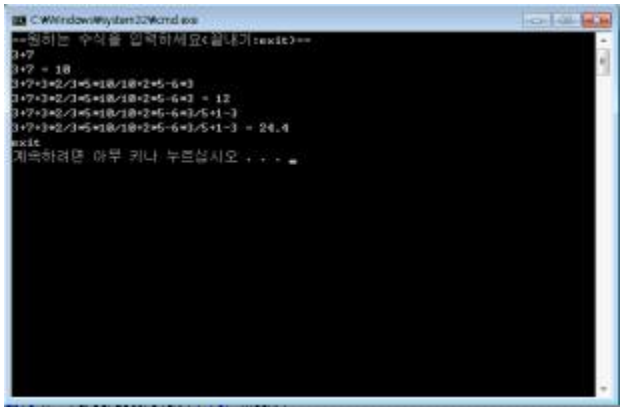
3. [난이도 中] 숫자로 하는 빙고 게임. 5행 5열의 숫자판을 1~25의 숫자로 중복 되지 않게 채우고, 컴퓨터와 빙고 게임을 한다. 행 또는 열, 대각선으로 5개의 숫자를 지우면 라인 1개를 완성한 것이고, 이러한 라인이 5개가 되면 승리한다. 게임은 한번 실행으로 사용자가 원하는 만큼 할 수 있으며, 사용자가 종료 명령을 내리기 전까지 종료 되어서는 안 된다. 컴퓨터의 숫자판은 사용자에게 보이지 않으며, 승부가 결정나면, 그때 보여준다. 그 외의 다양한 기능을 작성자가 임의로 추가 할 수 있으며, 그에 따른 내용을 보고서에 명시하여야 한다.

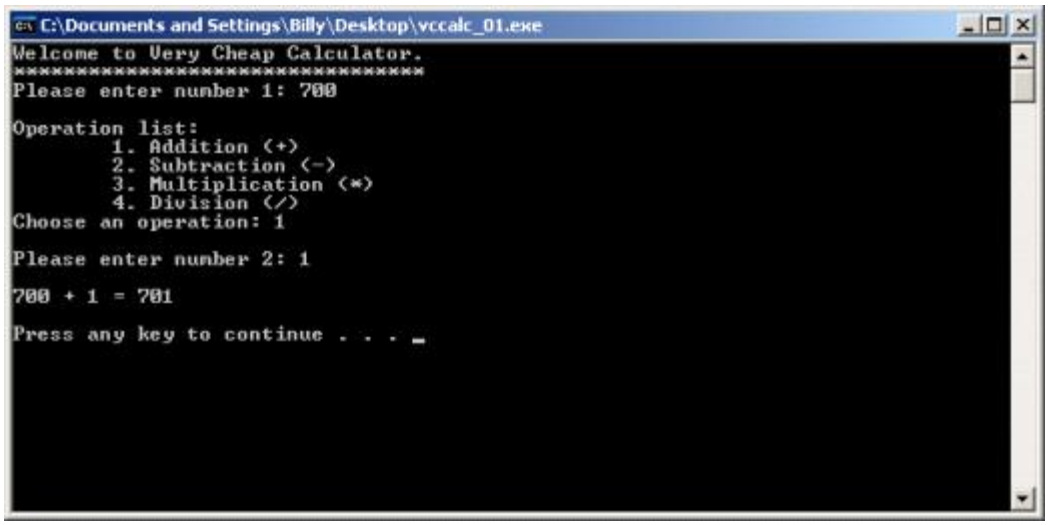
4. [난이도 中] 타자 연습 프로그램을 작성하시오. 임의의 문장을 화면에 보여주고, 사용자가 보여준 문장을 입력하고 엔터를 누르면, 사용자의 타수와 정확도를 보여주는 프로그램이다. 프로그램은 단문 연습과 장문 연습 두가지로 나누어 지며, 단문은 1개의 문장을 치면서 연습하는 기능이고, 장문은 1개 단락 정도의 예문을 보며 해당 문장을 치는 기능이다. 이때 각각 보여주는 문장은 파일을 이용하여 저장토록 한다. 저장된 파일은 1개 이상이어야 하며, 저장된 내용은 작성자 임의대로 작성토록 한다. 그 외의 다양한 기능을 작성자가 임의로 추가 할 수 있으며, 그에 따른 내용을 보고서에 명시하여야 한다.



5. [난이도 上] 산성비 타자 연습 프로그램을 작성하시오. 임의의 단어가 화면 상단에서 내려오면, 사용자는 그 단어를 빨리 입력하여 화면에서 지우는 프로그램이다. 단어가 화면 하단에 닿으면 사용자의 에너지가 줄어들고, 모든 에너지가 바닥나면 게임이 종료되면서 성적이 출력된다. 게임의 난이도는 모두 10단계 이상으로 하고, 각 단계별로 단어의 출현 빈도와 단어의 길이, 단어의 떨어지는 속도가 달라야 하며, 난이도가 높아질수록 빈도와 길이, 속다가 증가한다. 그 외의 다양한 기능을 작성자가 임의로 추가 할 수 있으며, 그에 따른 내용을 보고서에 명시하여야 한다.

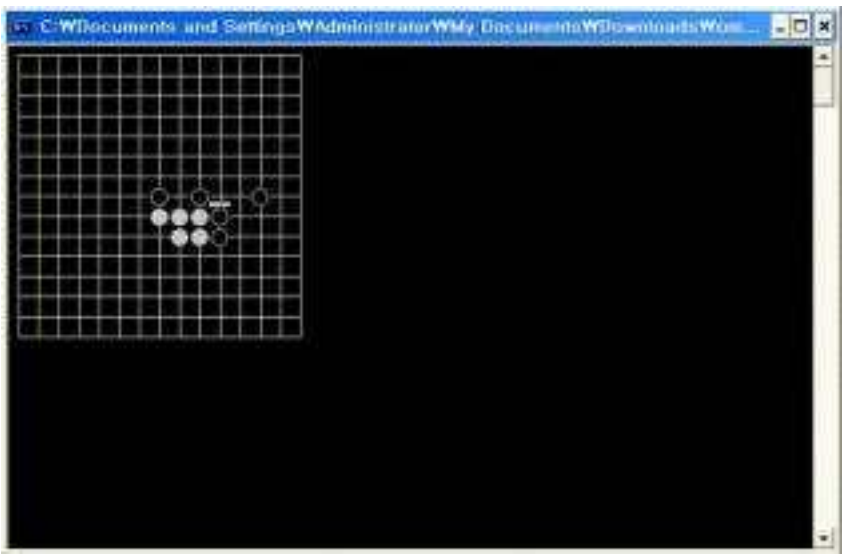
6. [난이도 中] 계산기 프로그램을 만드시오. 계산기 프로그램은 키보드를 통하여 식을 입력한 후 엔터를 입력하면 그 결과가 나오도록 하시오. 연산 결과를 출력할때에는 앞에 Result01 과 같은 형식으로 번호를 붙여 출력하시오. 추후 연산을 입력할 때 Result01 + 3 과 같이 입력하는 경우 Result01과 함께 출력되었던 결과값이 다시 연산에 사용되어 계산되어야 한다. 계산기는 +(더하기), -(빼기), *(곱하기), /(나누기)는 물론 ^(승 또는 거듭제곱), ()(괄호) 등의 다양한 연산을 수행할 수 있어야 하며, 연산의 순서는 연산자 우선순위를 따르도록 하되 괄호의 우선순위를 가장 높게 처리하시오. 또한, 추가적으로 sin, cos, tan 등의 삼각함수등을 추가로 구현하는 등의 기존의 계산기에서 계산할 수 있는 것들을 개인의 역량과 아이디어에 따라 추가하는 것을 권장함.

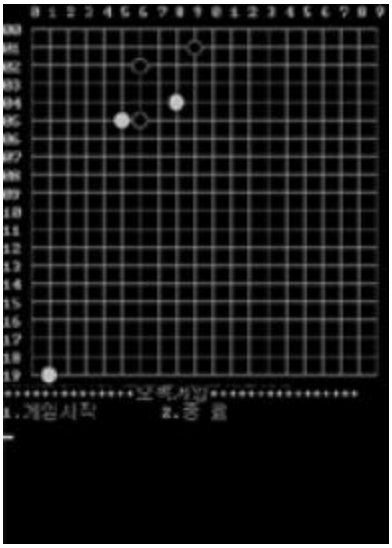


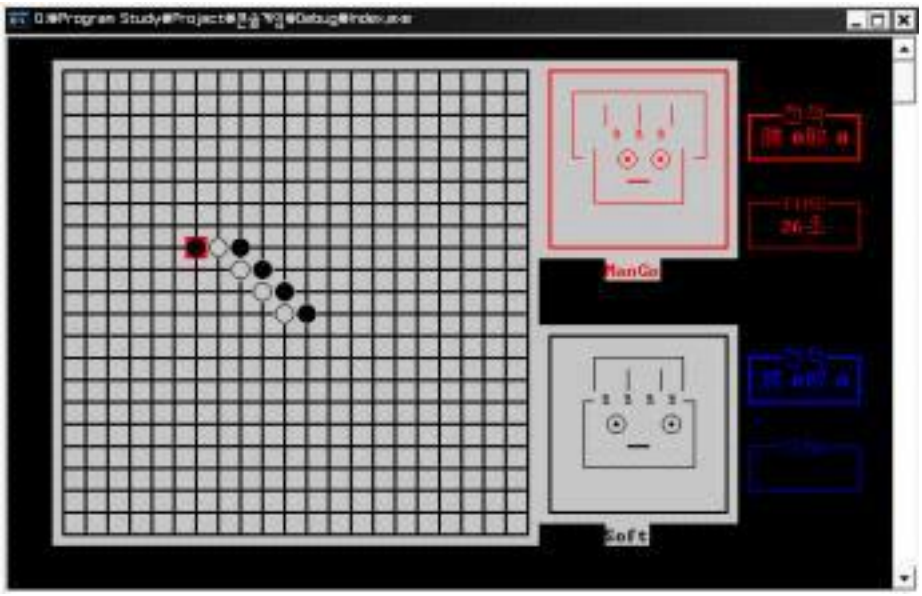


7. [난이도 中] 자판기 프로그램을 만드시오. 자판기 프로그램은 동전 또는 지폐를 넣고 자판기에서 판매하는 다양한 상품을 구매하는 형식으로 만드시오. 판매하는 제품과 가격은 각 개인이 선택하되, 프로그램이 시작하면 각 판매 상품마다의 수량이 고정되어 있어야 한다. 특정 상품의 재고가 없는 경우 화면에 재고가 없음을 표시하시오. 자판기 판매 매출 정보를 확인 할 수 있어야 하며, 자판기를 초기화 하는 기능도 추가하시오. 구현의 예는 아래의 그림을 참고하시오.

8. [난이도 上] 오목 프로그램을 만드시오. 사용자 2인이 함께 플레이 할 수 있는 오목프로그램을 만들되 키보드를 이용하여 플레이 할 수 있도록 작성하시오. 단, 오목 프로그램은 사용자 2인 중 한명이 승리하는 상황이면 승리자가 누구인지 화면에 표시하고 게임을 다시 할지 물어보도록 작성하시오. 게임을 진행하는 화면의 예는 아래의 그램을 참조하시오.







9. [난이도 中] *string.h*에서 제공하는 함수들을 구현한 *my_string.h*를 구현하고, 구현한 함수가 정상적으로 동작하는지 테스트하는 프로그램을 작성하시오. *string.h*에서 제공하는 함수의 리스트는 아래와 같으며, 이중 최소 8개 이상을 반드시 구현하시오.

함수	설명
복사	
void * **memcpy** (void * destination, const void * source, size_t num);	source가 가리키는 곳 부터 num바이트 만큼을 destination이 가리키는 곳에 복사한다.
void * **memmove** (void * destination, const void * source, size_t num);	source가 가리키는 곳 부터 num바이트 만큼을 destination이 가리키는 곳으로 옮긴다.
char * **strcpy** (char * destination, const char * source);	source를 destination에 복사한다.
char * **strncpy** (char * destination, const char * source, size_t num);	source에서 destination으로 처음 num개의 문자들을 복사한다.
병합	
char * **strcat** (char * destination, const char * source);	source를 destination뒤에 붙인다.
char * **strncat** (char * destination, char * source, size_t num);	source에서 destination뒤에 처음 num개의 문자들을 붙인다.
비교	
int **memcmp** (const void * ptr1, const void * ptr2, size_t num);	ptr1이 가리키는 처음 num바이트의 데이터와 ptr2가 가리키는 처음 num바이트의 데이터를 비교한다.
int **strcmp** (const char * str1, const char * str2);	str1과 str2를 비교한다.
int **strcoll** (const char * str1, const char * str2);	strcmp와 비슷하지만 LC_COLLATE에 정의되어 있는 방식에 따라 해석 된 후 비교한다.
int **strncmp** (const char * str1, const char * str2, size_t num);	str1의 처음 num개의 문자를 str2의 처음 num개의 문자와 비교한다.
size_t **strxfrm** (char * destination, const char * source, size_t num);	source를 현재 지역 정보에 따라 문자열을 변환한 후 변환한 문자열의 처음 num개 문자를 destination에 복사한다.
탐색	
void * **memchr** (const void * ptr, int value, size_t num);	ptr이 가리키는 메모리의 처음 num바이트 중에서 처음으로 value와 일치하는 값(문자)의 주소를 반환한다.
char * **strchr** (const char * str, int character);	str에서 처음으로 character와 일치하는 문자의 주소를 반환한다.
size_t **strcspn** (const char * str1, const char * str2);	str2에 들어있는 문자들 중 str1에 들어있는 문자와 일치하는 것이 있다면 첫 번째로 일치하는 문자까지 읽어들인 수를 반환한다.
char * **strpbrk** (const char * str1, const char * str2);	str1에서 str2에 들어 있는 문자들을 찾아 str2의 문자들 중 str1의 문자들과 첫 번째로 일치하는 문자의 주소를 반환한다.
char * **strrchr** (const char * str, int character);	str에서 마지막으로 character와 일치하는 문자의 주소를 반환한다.
size_t **strspn** (const char * str1, const char * str2);	str2의 문자들만을 포함하고 있는 str1의 처음 부분의 길이를 구한다.
char * **strstr** (const char * str1, const char * str2);	str1에서 str2를 검색하여 가장 먼저 나타나는 곳의 위치를 반환한다.
char * **strtok** (char * str, const char * delimiters);	str1을 delimiters의 문자들로 분리한다.
기타	
void * **memset** (void * ptr, int value, size_t num);	ptr이 가리키는 메모리의 처음 num바이트를 value 값(문자)으로 채운다.
char * **strerror** (int errnum);	errnum(보통 errno)을 해석한 뒤 그에 해당하는 에러 문자열의 포인터를 반환한다.
size_t **strlen** (const char * str);	str의 길이를 반환한다.

<출처 : 위키백과 http://ko.wikipedia.org/wiki/String.h>

- 구현한 my_string.h의 작성 난이도와 완성도, 보고서 작성 등에 따라 차등적으로 점수를 줄 예정임.

- 최소 8개 이상의 함수를 구현하여야 하며, 구현한 함수의 수가 모자를 경우 감정이 되면

추가하여 8개 이상의 함수를 구현하는 경우 추가 점수를 배정함. 이 때 난이도가 높은 8개의 함수를 기본으로 채점하고, 상대적으로 낮은 난이도의 함수를 추가 점수를 배정함.

- 작성한 *my_string.h*의 함수 이름은 기존 *string.h*의 함수 이름 앞에 "*my_*"를 붙여 구분한다. 예를들어, *string.h*의 *strlen()* 함수를 *my_string.h*에 구현하는 경우 *my_strlen()*으로 작성한다.

- 구현한 *my_string.h*와 기존의 *string.h*의 함수의 기능 비교를 위한 별도의 프로그램을 작성하여 각 헤더파일의 두 함수가 동일한 기능을 수행함을 증명해야 한다.

보고서 작성 예

my_string.h의 구현

20212021 홍길동

대구가톨릭대학교 컴퓨터소프트웨어학부

1. 서 론

보고서 전반에 대한 요약과 순서에 대해 설명한다.

2. 관련 연구

프로젝트 수행과 관련한 기타 이론이나 내용들을 정리한다.

3. 프로그램 설계

프로젝트를 수행하면서 프로그램을 구성하기 위한 계획과 기획, 구성들에 대해 설명한다.

4. 프로그램 구현

실제 프로그램을 구현한 내용과 기술들을 설명한다. (소스코드의 일부분을 인용가능)

5. 프로그램의 동작과 결과

프로그램의 동작 결과를 보여준다. (string.h와 my_string.h의 함수가 동일한 기능을 수행함을 보이는 프로그램의 실행 화면을 캡쳐하여 포함할 것)

6. 결과 분석 및 토의

5장의 동작 결과가 정상적으로 수행된 것인지.. 이상은 없는지.. 완벽한 것인지.. 하나 하나 면밀히 검토하여 분석하고 토의한다.

7. 결론

보고서의 결론을 맺고, 프로젝트를 하면서 느낀점과 향후 자신의 프로그래밍에 대한 짧은 소견을 밝힌다.

8. 참고문헌

[1] 안기수, “명품 C언어프로그래밍”, 생능출판사, 2013.

[2] Al Kelley, Ira Pohl 원저, 김명호 역, “A Book on C 한국어판”, 홍릉과학출판사, 1998.

[3] Keyword Search the Single INIX Specification,
http://www.opengroup.org/onlinepubs/007908799/index.html

[4] Teach Yourself C in 21 Days, https://kldp.org/files/c+ in+ 21+ days.pdf

9. 별첨 (소스코드와 주석)

1) my_string.h

```
#include <stdio.h>

int my_strlen(const char *); // 함수 원형 : strlen() 함수의 구현버전, 문자열의 길이를 계산하는 함수
char* my_strcpy(char*, const char*); // 함수 원형 : strcpy() 함수의 구현 버전, 문자열을 복사하는 함수
char* my_strncpy(char*, const char*, size_t); // 함수 원형 : strncpy() 함수의 구현 버전, 일정 크기 만큼의 문자열을 복사하는 함수

int my_strlen(const char *str) // my_strlen()의 정의, strlen() 함수의 구현버전, 문자열의 길이를 계산하는 함수
{
	int n = 0;
	return n;
}
```

2 TermProject.c

```
#include <stdio.h> // 표준 입출력 함수 사용을 위한 헤더파일 추가
#include <string.h> // 문자열을 다루는 함수 사용을 위한 헤더파일 추가
#include "my_string.h" // string.h 의 함수들을 직접 구현한 헤더파일

int main(int argc, char* argv[]) // 메인 함수 시작
{
	return 0; // 프로그램 정상 종료
}
```

(소스코드는 8포인트 크기로 붙인다.)